Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Qt Pencere ve Düzen Yönetimi

Bölüm 5 – Hafta 5

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 5. Hafta

Bu Haftanın Planı

1. Ünite 1: Düzen Yöneticileri
2. Ünite 2: Ana Pencere Tasarımı
3. Ünite 3: Diyalog Pencereleri
4. Ünite 4: Qt Designer ve UI Dosyaları
5. Özet ve Sonraki Hafta

Ünite 1: Düzen Yöneticileri

Neden Düzen Yöneticisi?

Problem

Widget'ları piksel koordinatlarıyla yerleştirmek:

- Farklı ekran çözünürlüklerinde bozulur
- Pencere boyutu değiştiğinde widget'lar sabit kalır
- Bakımı zordur

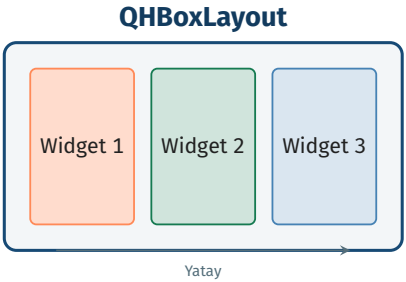
Çözüm: Layout Manager

Düzen yöneticisi, widget'ları otomatik olarak konumlandırır ve pencere boyutu değiştiğinde yeniden düzenler. Bölüm 4'te `QVBoxLayout`'u kısaca kullandık — şimdi derinleşiyoruz.

QHBoxLayout: Yatay Düzen

Widget'ları yatay olarak sıralar:

- `addWidget(w)` – widget ekle
- `addSpacing(px)` – sabit boşluk
- `addStretch()` – esnek boşluk
- Soldan sağa sırayla yerleştirir



OBYS Kullanımı

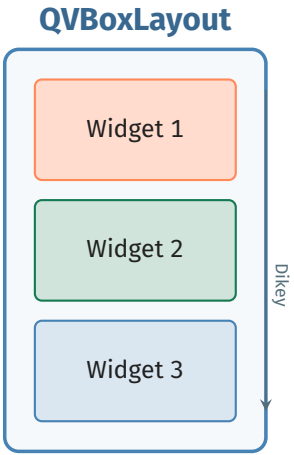
Form butonlarını yan yana dizmek için ideal:
`[Kaydet] [Temizle] [İptal]`

QVBoxLayout: Dikey Düzen

Widget'ları dikey olarak sıralar:

- `addWidget(w)` – widget ekle
- `addSpacing(px)` – sabit boşluk
- `addStretch()` – esnek boşluk
- Yukarıdan aşağıya sırayla yerleştirir

Bölüm 4'ten hatırla:
OBYS kayıt formunda `QVBoxLayout` ile başlık, form ve butonları alt alta dizmiştik.



İç İçe Düzenler

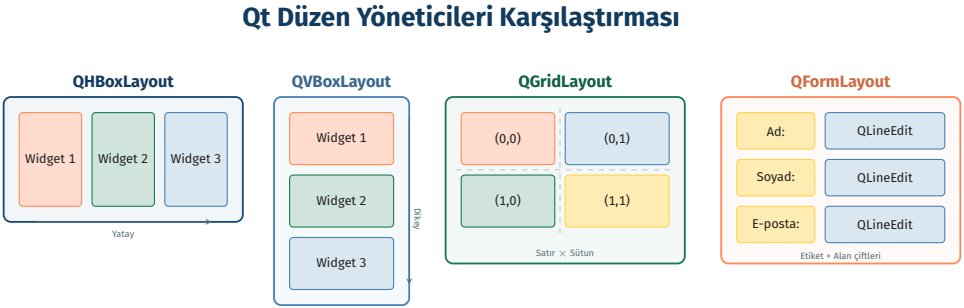
Anahtar Fikir

Düzenleri iç içe yerleştirerek karmaşık arayüzler oluşturabilirsin. `addLayout()` ile bir düzeni başka bir düzenin içine ekle.

```
QVBoxLayout* anaDuzen = new QVBoxLayout(&pencere);

// Ust bolum: yatay butonlar
QHBoxLayout* ustDuzen = new QHBoxLayout();
ustDuzen->addWidget(btn1);
ustDuzen->addWidget(btn2);
anaDuzen->addLayout(ustDuzen);  // ic ice!

// Alt bolum: etiket
anaDuzen->addWidget(etiket);
```

VBox → [HBox → btn1, btn2] + etiket

Düzen Türleri Karşılaştırması



Dört temel düzen yöneticisi: HBox (yatay), VBox (dikey), Grid (ızgara), Form (etiket-alan çiftleri).

QGridLayout: Izgara Düzeni

Satır/sütun tabanlı yerleşim:

```
QGridLayout* grid = new QGridLayout()
    ;

// addWidget(w, satir, sutun)
grid->addWidget(w1, 0, 0);
grid->addWidget(w2, 0, 1);
grid->addWidget(w3, 1, 0);

// Hucre birlestirme (spanning)
// addWidget(w, satir, sutun,
//           satirSpan, sutunSpan)
grid->addWidget(genis, 2, 0, 1, 2);
```

Kullanım Alanları

- Hesap makinesi butonları
- Karmaşık form düzenleri
- Tablo benzeri arayüzler
- Hücre birleştirme gereken durumlar

QFormLayout: Form Düzeni

```
QFormLayout* form = new QFormLayout()
    ;

// addRow(etiket, widget)
form->addRow("Ad Soyad:", adGiris);
form->addRow("No:", numaraGiris);
form->addRow("Bolum:", bolumSecim);
```

Avantajları:

- Platform uyumlu hizalama
- macOS: sağa hizalı etiketler
- Windows/Linux: sola hizalı

QFormLayout vs QGridLayout

Form	Grid
Basit formlar	Karmaşık
2 sütun	N sütun
Platform uyumlu	Manuel
addRow()	addWidget()

QSizePolicy ve Stretch

QSizePolicy Türleri

Politika	Davranış
`Fixed`	Sabit boyut, değişmez
`Minimum`	En az kendi boyutu kadar
`Maximum`	En fazla kendi boyutu kadar
`Preferred`	İdeal boyutu tercih eder (varsayılan)
`Expanding`	Tüm boş alanı kaplar

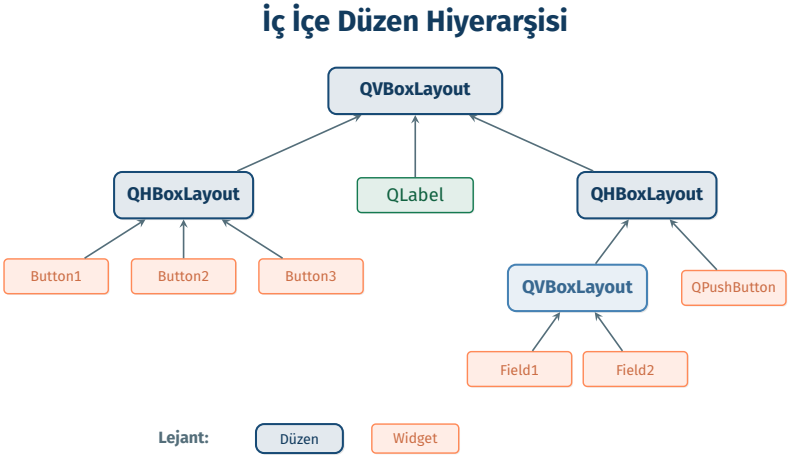
Stretch Faktörleri:

- `addWidget(w, stretch)`
- Oran belirler: 1:2:3
- `addStretch()` → boş esnek alan

Margins ve Spacing:

- `setContentsMargins(l,t,r,b)`
- `setSpacing(px)`
- Varsayılan: platforma bağlı

İç İçe Düzen Hiyerarşisi



Düzenleri iç içe yerleştirerek karmaşık arayüzler oluşturabilirsin.

Ünite 2: Ana Pencere Tasarımı

QWidget'tan QMainWindow'a

QWidget (Bölüm 4)

- Basit pencere
- Menü yok
- Araç çubuğu yok
- Durum çubuğu yok
- Her şeyi sen yönet

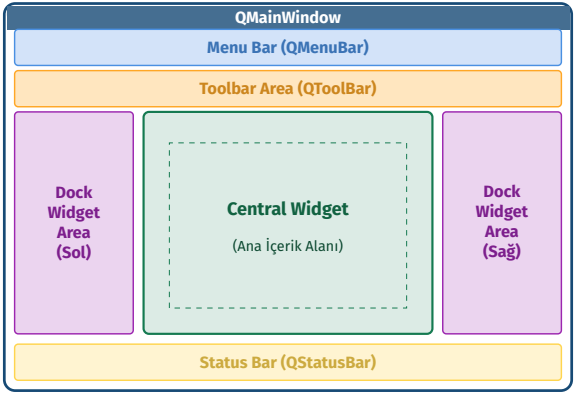
QMainWindow (Bölüm 5)

- Profesyonel pencere
- Menü çubuğu hazır
- Araç çubuğu desteği
- Durum çubuğu hazır
- Dock widget desteği

`QMainWindow` IS-A `QWidget` → tüm QWidget özelliklerini de taşır!

QMainWindow Mimarisi

QMainWindow Mimarisi



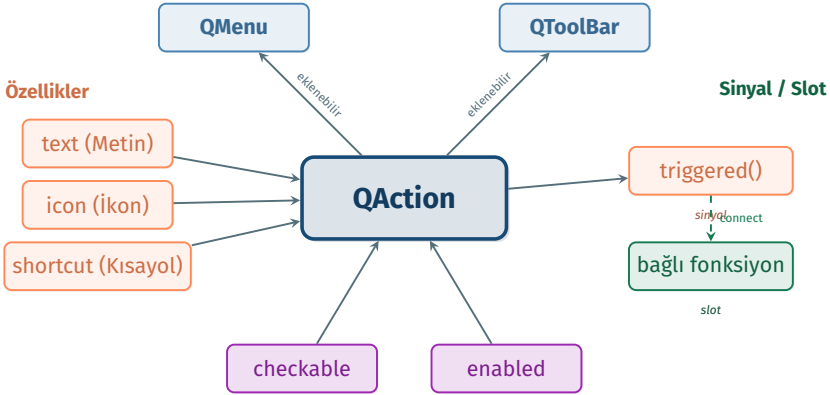
setCentralWidget()

```
class AnaPencere : public QMainWindow {
public:
    AnaPencere(QWidget* parent = nullptr)
        : QMainWindow(parent) {
        setWindowTitle("OBYS");
        resize(800, 600);

        // Merkezi widget
        QTextEdit* metin = new QTextEdit();
        setCentralWidget(metin);
    }
};
```

Dikkat

QMainWindow'a doğrudan `setLayout()` çağırma! Kendi iç düzeni var. Widget'ları merkezi widget'ın içine yerleştir.

QAction İlişki Diyagramı



QMenuBar ve QMenu

```
// Eylem olustur
QAction* kaydetEylem = new QAction("&Kaydet", this);
kaydetEylem->setShortcut(QKeySequence::Save);
kaydetEylem->setStatusTip("Belgeyi kaydet");

// Menu olustur
QMenu* dosyaMenu = menuBar()->addMenu("&Dosya");
dosyaMenu->addAction(yeniEylem);
dosyaMenu->addAction(acEylem);
dosyaMenu->addAction(kaydetEylem);
dosyaMenu->addSeparator();
dosyaMenu->addAction(cikisEylem);
```

& karakteri klavye kısayol harfini belirler: Dosya, Kaydet

Standart Kısayollar

QKeySequence Standartları

Enum	Windows/Linux	macOS
`QKeySequence::New`	Ctrl+N	Cmd+N
`QKeySequence::Open`	Ctrl+O	Cmd+O
`QKeySequence::Save`	Ctrl+S	Cmd+S
`QKeySequence::Quit`	Ctrl+Q	Cmd+Q
`QKeySequence::Copy`	Ctrl+C	Cmd+C
`QKeySequence::Paste`	Ctrl+V	Cmd+V
`QKeySequence::Cut`	Ctrl+X	Cmd+X

Platform bağımsız kısayollar → tek kod, her yerde doğru çalışır!

QToolBar

```
QToolBar* aracCubugu = addToolBar("Ana");

// Ayni QAction'lari ekle (menu ile paylasim!)
aracCubugu->addAction(yeniEylem);
aracCubugu->addAction(kaydetEylem);
aracCubugu->addSeparator();
aracCubugu->addAction(kesEylem);
aracCubugu->addAction(kopyalaEylem);
```

QAction Paylaşımı

Aynı `QAction` nesnesi hem menüde hem araç çubuğunda kullanılır. Birini devre dışı bırakırsan (`setEnabled(false)`), diğeri de otomatik olarak devre dışı kalır!

QStatusBar

```
// Gecici mesaj (5 saniye)
statusBar()->showMessage("Kaydedildi", 5000);

// Kalici widget
QLabel* kayitSayisi = new QLabel("Kayit: 0");
statusBar()->addPermanentWidget(kayitSayisi);
```

Geçici Mesajlar:

- `showMessage(msg, ms)`
- Belirli süre sonra kaybolur
- İşlem bildirimleri

Kalıcı Widget'lar:

- `addPermanentWidget(w)`
- Her zaman görünür
- Durum göstergeleri

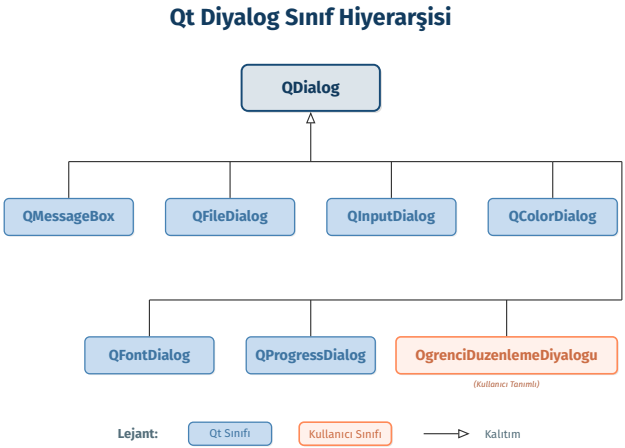
QDockWidget

```
QDockWidget* dock = new QDockWidget("Gezinme", this);

dock->setAllowedAreas(
    Qt::LeftDockWidgetArea | Qt::RightDockWidgetArea);

dock->setFeatures(
    QDockWidget::DockWidgetMovable |
    QDockWidget::DockWidgetClosable |
    QDockWidget::DockWidgetFloatable);

dock->setWidget(listeWidget);
addDockWidget(Qt::LeftDockWidgetArea, dock);
```

Dock widget'lar sürüklenebilir, yüzdürülebilir ve kapatılabilir panellerdir. OBYS'ta gezinme paneli olarak kullanacağız.

Ünite 3: Diyalog Pencereleri

QDialog Sınıf Hiyerarşisi



Özel QDialog Oluşturma

```
class OgrenciDiyalogu : public QDialog {
public:
    OgrenciDiyalogu(QWidget* parent = nullptr)
        : QDialog(parent) {
        setWindowTitle("Ogrenci Duzenle");
        QFormLayout* form = new QFormLayout(this);
        adGiris_ = new QLineEdit();
        form->addRow("Ad Soyad:", adGiris_);
        auto* btn = new QDialogButtonBox(
            QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
        form->addRow(btn);
        connect(btn, &QDialogButtonBox::accepted,
                this, &QDialog::accept);
        connect(btn, &QDialogButtonBox::rejected,
                this, &QDialog::reject);
    }
    QString ad() const { return adGiris_->text(); }
private:
    QLineEdit* adGiris_;
};
```

accept() / reject() ve exec()

```
OgrenciDiyalogu diyalog(this);

// exec() -> modal, sonuc doner
if (diyalog.exec() == QDialog::Accepted) {
    // Kullanici "OK" dedi
    QString ad = diyalog.ad();
    qDebug() << "Eklendi:" << ad;
} else {
    // Kullanici "Iptal" dedi veya Escape
    qDebug() << "Iptal edildi";
}
```

QDialogButtonBox

Standart butonları platforma uygun sırayla yerleştirir. macOS'ta "Cancel" solda, Windows'ta sağda olur.

QMessageBox

```
// Bilgi
QMessageBox::information(this,
    "Bilgi", "Kayit basarili!");

// Uyari
QMessageBox::warning(this,
    "Uyari", "Numara mevcut!");

// Hata
QMessageBox::critical(this,
    "Hata", "Baglanti hatasi!");
```

```
// Soru (donus degeri onemli!)
auto sonuc = QMessageBox::question(
    this, "Onay",
    "Silmek istediginize emin misiniz?",
    QMessageBox::Yes | QMessageBox::No,
    QMessageBox::No);

if (sonuc == QMessageBox::Yes) {
    // Sil
}
```

Statik metodlar tek satırda diyalog açar ve sonucu döndürür.

QFileDialog ve QInputDialog

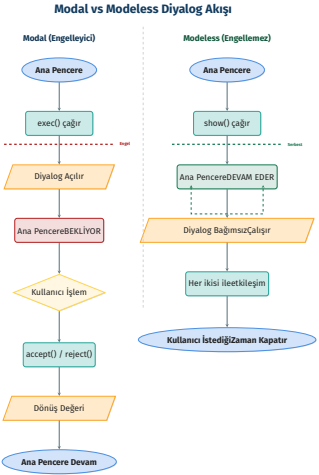
QFileDialog:

```
// Dosya ac
QString yol =
  QFileDialog::getOpenFileName(
    this, "Dosya Ac", "",
    "Metin (*.txt);;Tumu (*)");

// Dosya kaydet
QString kayitYolu =
  QFileDialog::getSaveFileName(
    this, "Kaydet",
    "liste.txt");
```

QInputDialog:

```
// Metin girisi
bool ok;
QString isim =
  QInputDialog::getText(
    this, "Ad",
    "Ogrenci adi:",
    QLineEdit::Normal,
    "Ayse Yilmaz", &ok);

// Sayi girisi
int no = QInputDialog::getInt(
    this, "No",
    "Ogrenci no:",
    1000, 1000, 9999, 1, &ok);
```

Modal vs Modeless



exec() vs show()

Modal: exec()

```
// Engelleyici (blocking)
QDialog dlg(this);
int sonuc = dlg.exec();
// Bu satir dlg kapaninca calisir
```

- Ana pencere kilitlenir
- Sonuç hemen alınır
- Stack'te oluşturulabilir
- Veri girişi, onay

Modeless: show()

```
// Engellemez (non-blocking)
auto* dlg = new QDialog(this);
dlg->setAttribute(
    Qt::WA_DeleteOnClose);
dlg->show();
// Kod hemen devam eder!
```

- Ana pencere aktif kalır
- `finished()` sinyali ile takip
- Heap'te oluşturulmalı
- Araç panelleri, loglar

Ünite 4: Qt Designer ve UI Dosyaları

Qt Designer Nedir?

Görsel Form Editörü

- Sürükle-bırak widget yerleştirme
- **Widget Box**: Kullanılabilir widget'lar
- **Object Inspector**: Nesne hiyerarşisi
- **Property Editor**: Widget özellikleri
- **Signal/Slot Editor**: Bağlantılar

Çıktı: `.ui` dosyası (XML formatı)

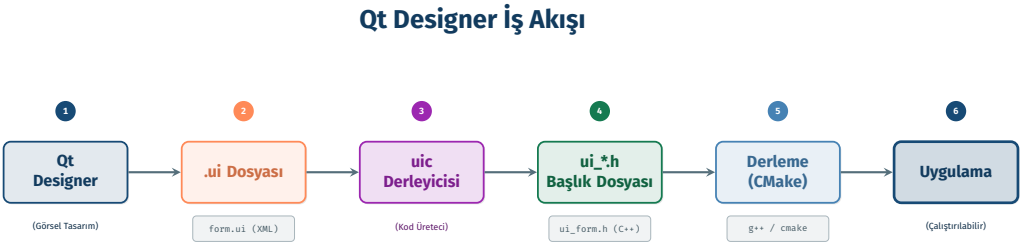
Ne Zaman Kullanmalı?

- Karmaşık formlar
- Hızlı prototipleme
- Tasarımcı-geliştirici iş birliği

Ne Zaman Kod Yazmalı?

- Dinamik arayüzler
- Programatik mantık
- Basit pencereler

Designer → Uygulama İş Akışı



Qt Designer → `.ui` (XML) → `uic` derleyicisi → `ui_*.h` (C++ kodu) → derleme → uygulama

Önerilen: Tek Kalıtım (Single Inheritance)

```
#include "ui_kayit_formu.h"

class KayitFormu : public QWidget {
public:
    KayitFormu(QWidget* parent = nullptr)
        : QWidget(parent) {
        ui.setupUi(this);
        connect(ui.kaydetButon, &QPushButton::clicked,
                this, &KayitFormu::kaydet);
    }
private:
    Ui::KayitFormu ui;  // UI sinifi uye olarak
};
```

Uyarı

`ui_*.h` dosyalarını **asla** elle düzenleme! Otomatik üretilir.

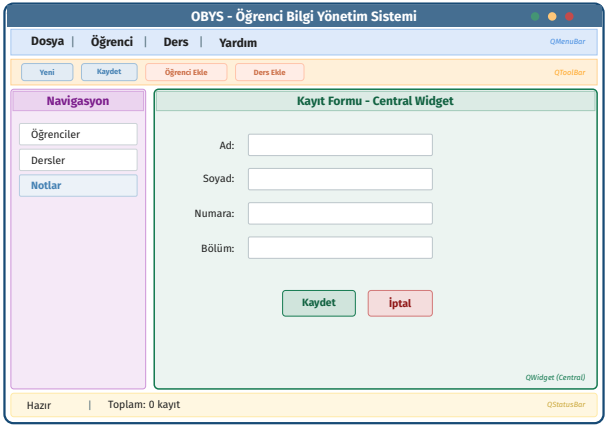
CMake ile AUTOUIC

```
cmake_minimum_required(VERSION 3.16)
project(OBYS LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_AUTOMOC ON)     # moc otomatik
set(CMAKE_AUTOUIC ON)     # uic otomatik

find_package(Qt6 REQUIRED COMPONENTS Widgets)

add_executable(obys
    main.cpp
    obys_ana_pencere.cpp
    obys_ana_pencere.h
    kayit_formu.ui         # .ui dosyasi
)
target_link_libraries(obys PRIVATE Qt6::Widgets)
```

`AUTOUIC ON` → CMake, `.ui` dosyalarını otomatik bulup `uic` ile derler.

OBYS Ana Pencere: Tüm Bölümün Birleşimi

OBYS Ana Pencere Yapısı



OBYS Ana Pencere: Bileşenler

Ünite 1 – Düzenler:

- QFormLayout: kayıt formu
- QVBoxLayout: ana düzen
- QHBoxLayout: buton satırı

Ünite 2 – Ana Pencere:

- QMainWindow: iskelet
- QMenuBar: 4 menü
- QToolBar: sık eylemler
- QStatusBar: kayıt sayısı
- QDockWidget: gezinme

Ünite 3 – Diyaloglar:

- QMessageBox::about()
- Doğrulama uyarıları
- Onay diyalogları

Ünite 4 – Designer:

- .ui dosya entegrasyonu
- CMake AUTOUIC
- Kod-tasarım ayrımı

Özet ve Sonraki Hafta

Bu Hafta Öğrendiklerimiz

1. **Düzen Yöneticileri**: QHBoxLayout, QVBoxLayout, QGridLayout, QFormLayout, iç içe düzenler, QSizePolicy
2. **QMainWindow**: 5 bölge mimarisi, setCentralWidget(), menü/araç/durum çubuğu, QDockWidget
3. **QAction**: Metin + ikon + kısayol, menü ve toolbar paylaşımı
4. **Diyaloglar**: QDialog, accept/reject, exec/show, QMessageBox, QFileDialog, QInputDialog
5. **Modal vs Modeless**: exec() engeller, show() engellemez
6. **Qt Designer**: .ui dosyası, uic, CMake AUTOUIC

OBYS Yol Haritası

Tamamlanan

- Bölüm 1–3: STL ile veri katmanı (vector, map, algoritmalar)
- Bölüm 4: Qt temelleri (QObject, sinyal/slot, widget'lar)
- Bölüm 5: Pencere ve düzen yönetimi, OBYS ana penceresi

Sonraki Hafta – Bölüm 6

Qt Model/View Mimarisi:

- QTableView ile öğrenci verilerini tablo görünümünde gösterme
- QStandardItemModel ve özel model oluşturma
- Sıralama, filtreleme (QSortFilterProxyModel)
- OBYS: öğrenci listesi tablo görünümü

Pratik Öneriler

1. OBYS ana penceresine yeni menü öğeleri ekle
2. Farklı düzen yöneticilerini dene: ızgara hesap makinesi
3. Özel bir diyalog yaz: ders ekleme formu
4. Qt Designer ile .ui dosyası oluştur ve C++ ile entegre et
5. Dock widget'ı kapatıp açan bir menü öğesi ekle

Sorular?